

Finding Change Points by Optimization

Replacing the heuristic cascade with a single objective

Yuli Tshuva

September 2026

1 The problem

The old change point detector was a chain of local decisions. It looked at the sign of the derivative, then filtered by prominence, then merged points that were close together, then dropped points that had the same direction on both sides. Each step had its own threshold: `SEGMENT_PERCENTAGE`, `CHANGE_THRESHOLD`, `MIN_DISTANCE_BETWEEN_FEATURE_POINTS`, a prominence of 5% of the amplitude, a merge rule at 1% of the amplitude, a smoothing window.

Two things are wrong with this. First, there are six knobs and no principled way to set any of them. Second, and worse, the steps run in order, so the result depends on that order. A point dropped early can never come back. There is no sense in which the answer is the best answer — it is just the answer that falls out of this particular pipeline.

2 The idea

Say instead what a *good* segmentation is, and then find the best one.

Let the sequence have n points. A segmentation is a set of cut positions $0 = t_0 < t_1 < \dots < t_K = n$. Give every segment a cost $c(a, b)$, meaning “how badly does a simple model explain the piece $y_{a:b}$ ”. Then solve

$$\min_{K, t_1 < \dots < t_{K-1}} \sum_{k=0}^{K-1} c(t_k, t_{k+1}) + \beta K. \quad (1)$$

The first term wants many short segments, because short pieces are easy to explain. The term βK pushes back: every extra cut costs β . The balance between them decides how many change points there are.

This is solved *exactly*, not approximately, by dynamic programming. Let $F(t)$ be the best cost for the first t points. Then

$$F(t) = \min_{s < t} [F(s) + c(s, t) + \beta], \quad F(0) = 0, \quad (2)$$

and the cuts are read back by following which s won at each t . This is $O(n^2)$, and it runs in about 0.1 seconds for $n = 1000$.

Everything the old code did in a cleanup pass now lives inside Equation (2) as a constraint. A minimum segment length becomes $s \leq t - m$ in the search. A fixed number of change points becomes a second table $C(k, t)$ instead of a penalty. A rule like “no two neighbouring segments may go the same way” becomes an extra state. In every case the constraint is respected *and* the answer is still globally optimal — which the order-dependent merging passes could never promise.

3 What is a segment cost?

Two choices are implemented.

Straight-line cost. Fit a least-squares line to the piece and take the leftover error:

$$c(a, b) = \min_{\alpha, \gamma} \sum_{t=a}^{b-1} (y_t - \alpha - \gamma t)^2. \quad (3)$$

This says “a segment is a trend”, which is exactly what a node in the graph is meant to be.

Feature cost. The distance function compares segments through a feature vector ϕ , using weights w that were already tuned. Four of those features — mean value, mean slope, mean absolute slope, mean curvature — are simply the averages of four pointwise signals. Write that pointwise vector as $\psi(t)$ and ask each segment to be flat in *that* space:

$$c(a, b) = \sum_{t=a}^{b-1} \|w \odot (\psi(t) - \bar{\psi}_{a:b})\|^2, \quad \bar{\psi}_{a:b} = \frac{1}{b-a} \sum_{t=a}^{b-1} \psi(t). \quad (4)$$

Now the segmentation is chosen in the same space, with the same weights, that the similarity metric later measures. That is the honest meaning of “change point detection driven by the metric”.

Both costs are sums of squares, so with prefix sums of y, y^2, ty, t, t^2 (and of ψ, ψ^2) each $c(a, b)$ is computed in constant time. That is why the whole thing is fast.

4 The one remaining knob

Only β is left. The textbook choice is a BIC penalty, $\beta = 2\hat{\sigma}^2 p \log n$. On this data it fails: the sequences are smoothed with a window of 5 and stretched to 1000 points, so the estimated noise $\hat{\sigma}^2$ is almost zero, β is almost zero, and the method returns sixty or more change points.

So β is set relative to the sequence itself:

$$\beta = \beta_{\text{rel}} \cdot c(0, n). \quad (5)$$

In words: a new cut is worth making only if it removes at least β_{rel} of the error you would have with no cuts at all. This is a pure number, comparable across sequences, and it is the single quantity to tune against what we actually care about — agreement with human similarity judgements.

Figure 1 shows what it does. The result is stable: $K = 4$ holds over the whole range $\beta_{\text{rel}} \in [0.01, 0.05]$, so this is a broad plateau and not a knife edge.

UPS: one interpretable knob replaces the whole heuristic cascade

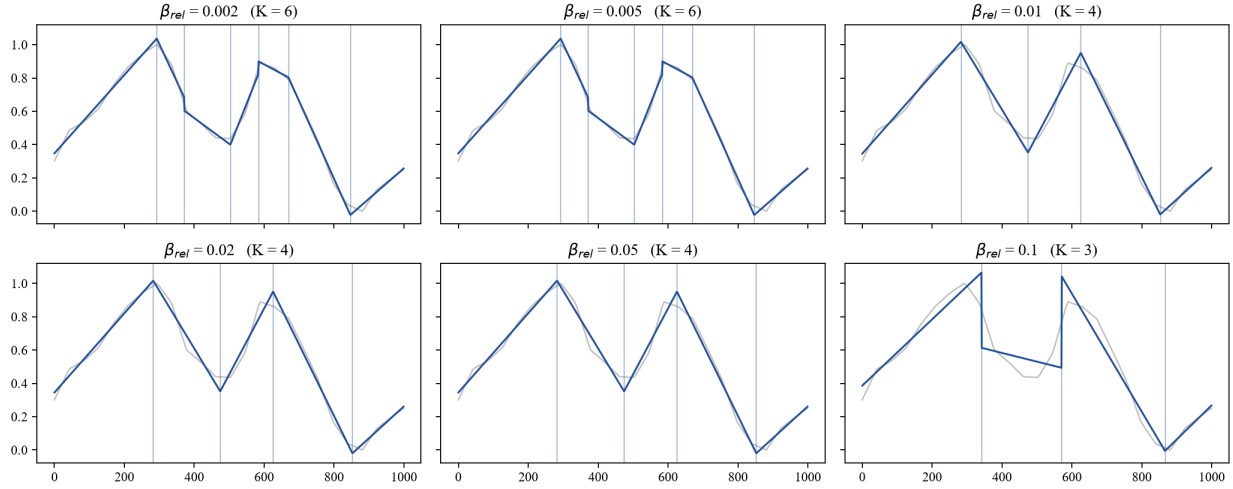


Figure 1: One interpretable knob replaces six thresholds. Small β_{rel} buys more segments, large β_{rel} buys fewer, and the middle range is flat.

5 How it turns out

Figure 2 shows what the method returns on six stocks: the sequence, with the change points marked on it.

The cuts land where a reader would put them — on the turns, at the tops and bottoms, and where a steady climb changes its rate. Nothing is placed inside a stretch that is already going one way, and no turn is left unmarked.

Table 1 compares the two methods as a number. For each stock the dynamic program was given the *same* number of change points the heuristic chose, so the comparison is fair. It removes between a third and four fifths of the leftover error, on the same budget.

Table 1: Reconstruction error at an equal number of change points.

Stock	K	Heuristic	Optimal	Error removed
AMZN	4	0.761	0.288	62%
SO	4	0.403	0.116	71%
UPS	4	1.475	0.993	33%
SBUX	3	7.014	1.490	79%
UNH	4	1.206	0.688	43%
SO (2018)	4	0.716	0.167	77%

Figure 3 is the same statement without picking a K . The blue curve is the lowest error reachable with K cuts — by construction nothing can go below it. The heuristic sits well above it.

6 What this does not yet show

Low reconstruction error is the dynamic program’s own objective, so it wins that by definition. It is *not* evidence that the similarity metric got better. That claim needs the triplet data from the human study: sweep

Change points from the optimal segmentation ($\beta_{rel} = 0.01$)

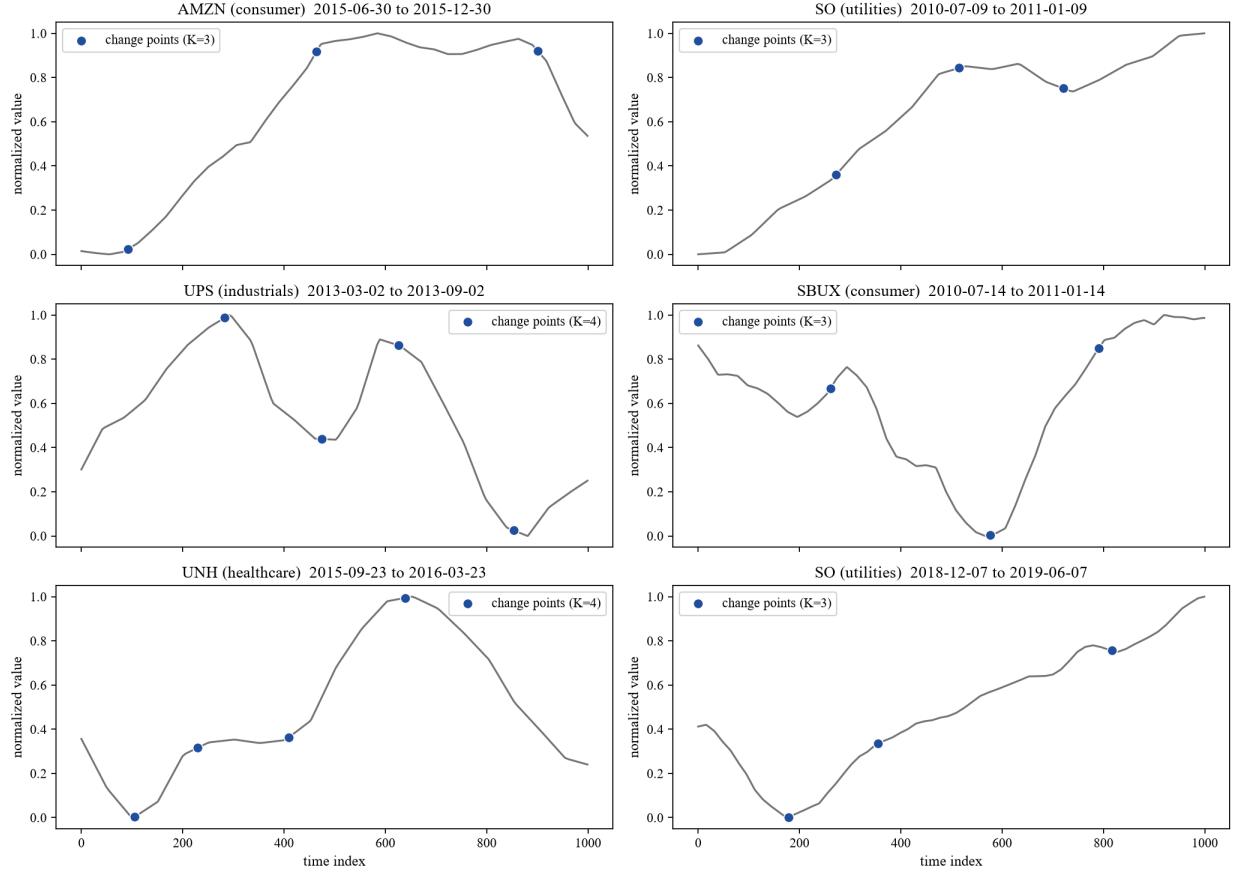


Figure 2: Six stocks with the change points found by the dynamic program at $\beta_{rel} = 0.01$.

For any budget K the DP attains the minimum achievable cost

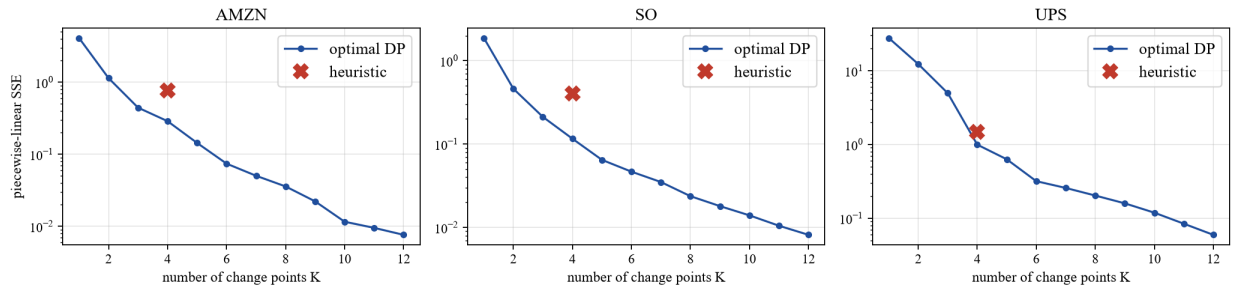


Figure 3: Best achievable error against the number of change points. The heuristic (red cross) is above the optimal curve at its own budget.

β_{rel} , measure agreement with human judgements, and compare against the heuristic. That is the next step, and it is the point at which the segmentation really becomes “optimized for the metric”.

7 Where the code is

- `cpd_opt.py` — the two costs, the penalized dynamic program, and the fixed- K version. The entry point `optimal_change_points()` returns boundaries in the same format as the old `change_points_detection()`, so it is a drop-in replacement.
- `demo_cpd_opt.py` — produces the three figures above and the table.